

Check Items			Example / Amplification
No.	Class	Description	
1	General	Rule 1 <code>“//”</code> comment style must not be used	Non-compliant example <pre>// TODO : Test this function</pre> Compliant example <pre>/* TODO : Test this function */</pre>
2		Rule 2 <code>“{”</code> and <code>“}”</code> should not be same statement line	Non-compliant example <pre>if(u1t_x == 2) { u1t_y = 4; }</pre> Compliant example <pre>if(u1t_x == 2) { u1t_y = 4; }</pre> Exception <pre>if(u1t_x == 2) { u1t_y = 4; } else { <- Exception u1t_y = 2; }</pre>
3		Rule 3 It should have one space between operand and operator	Non-compliant example <pre>u1t_x = (u1t_y+u1t_x)/u1t_a;</pre> Compliant example <pre>u1t_x = (u1t_y + u1t_x) / u1t_a;</pre>
4		Rule 4 Use one assignment per one line	Non-compliant example <pre>u1t_x = u1t_a; u1t_y = u1t_a; u1t_z = u1t_c;</pre> Compliant example <pre>u1t_x = u1t_a; u1t_y = u1t_b; u1t_z = u1t_c;</pre>
5		Rule 5 Hard-coded <code>“magic”</code> integer constants should be best avoided	Non-compliant example <pre>u1t_x = 63 - u1t_a;</pre> Compliant example <pre>#define FACTOR_A 63 u1t_x = FACTOR_A - u1t_a; const uint8_t FACTOR_B 43 u1t_y = FACTOR_B - u1t_a;</pre>

Check Items			Example / Amplification														
No.	Class	Description															
6	General	Rule 6 Ternary operator should be best avoided	Non-compliant example <pre>uint_x = (uint_a == 0) ? uint_y : uint_z;</pre> Compliant example <pre>if (uint_a == 0) { uint_x = uint_y; } else { uint_x = uint_z; }</pre>														
		Unused Code	Rule 7 A project shall not contain unreachable code	Non-compliant example <pre>uint8_t u1_getConvFactor(void) { const uint8_t CONVFACTORA = 104u; return ConvFactor; const uint8_t CCONVFACTORB = 94u; <- Unreachable Code }</pre>													
8		Rule 8 A project should not contain unused object definitions	An object is unused if the definition (and any declarations) can be removed, and the program still compiles.														
9	Literals and Constants	Rule 9 Octal constants shall not be used	An octal constant is a number in base 8 (octal numeral system) and is represented in code by prefixing it with 0 (zero). For example: 012 (octal) is equivalent to 10 in decimal.														
10		Rule 10 A “u” or “U” suffix shall be applied to all integer constants that are represented in an unsigned type	<table><tr><th>Constant</th><th>Type</th><th>Compliance</th></tr><tr><td>32767</td><td>int16_t</td><td>Compliant</td></tr><tr><td>0x7fff</td><td>int16_t</td><td>Compliant</td></tr><tr><td>32768</td><td>uint16_t</td><td>Non-compliant</td></tr><tr><td>32768u</td><td>uint16_t</td><td>Compliant</td></tr></table>	Constant	Type	Compliance	32767	int16_t	Compliant	0x7fff	int16_t	Compliant	32768	uint16_t	Non-compliant	32768u	uint16_t
Constant	Type	Compliance															
32767	int16_t	Compliant															
0x7fff	int16_t	Compliant															
32768	uint16_t	Non-compliant															
32768u	uint16_t	Compliant															
11	Declarations and Definitions	Rule 11 Function types shall be in prototype form with named parameters	Non-compliant example <pre>static uint16_t u2_func1(); <- Not in prototype form uint8_t u1_func2(uint8_t); <- parameter name not specified</pre> Compliant example <pre>extern uint32_t u4_func3(void); <- Specifies 0 parameters int16_t s2_func3(uint16_t u2_x);</pre>														
12	Initialization	Rule 12 The value of an object with automatic storage duration shall not be read before it has been set	Non-compliant example <pre>int32_t u4_func5(int32_t s4t_x) { uint32_t u4t_y; int32_t s4t_result; if (u4t_y == 0) { <- u4t_y has not been assigned a value u4t_result = (s4t_x * s4t_x); } else { u4t_result = s4t_x; } return s4t_result; }</pre>														

Check Items			Example / Amplification
No.	Class	Description	
13	The essential type model	Rule 13 Operands shall not be of an inappropriate essential type	Non-compliant example u4t_x && u4t_y <- Uses uint32_t with boolean operator !s2t_x <- Uses int16_t with boolean operator u4t_x << (u2t_a == u2t_b) <- Uses boolean in shift operation u4t_x >> -1 <- Shift magnitude uses signed type
14	Expressions	Rule 14 The precedence of operators within expressions should be made explicit	Non-compliant example u4t_x = u4t_a + u4t_b / u4t_c - u4t_d * u4t_e; Compliant example u4t_x = u4t_a + ((u4t_b / u4t_c) - (u4t_d * u4t_e));
15	Side Effects	Rule 15 Floating-point expressions shall not be tested for equality or inequality.	Floating-point numbers are an approximation, using a finite number of bits to represent real numbers leading to small rounding errors. Instead of directly testing for equality, use a tolerance-based approach to compare floating-point numbers. float f4t_a = 0.1f + 0.2f; <- stored as 0.30000001192092896 float f4t_b = 0.3f; <- stored as 0.29999998211860657 Non-compliant example if(f4t_a == f4t_b) { <- This can never be true // Do something } Compliant example #include <math.h> #define TOLERANCE 0.000001f if(fabs(f4t_a - f4t_b) < TOLERANCE) { // Do something }
16	Control Statement Expressions	Rule 16 A loop counter shall not have essentially floating type	Non-compliant example for (float f4t_cnt = 0.0f; f4t_cnt < 1.0f; f4t_cnt += 0.01f) { // Do something } float u4t_idx = 0.0f while (u4t_idx < 10.0f) { u4t_idx += 0.1f; }

Check Items			Example / Amplification
No.	Class	Description	
17	Control Statement Expressions	Rule 17 The controlling expression of an if statement and the controlling expression of an iteration-statement shall have essentially Boolean type	Non-compliant example <pre>if (u4t_x) { <- Expression is not boolean type } while (s2t_y) { <- Expression is not boolean type }</pre> Compliant example <pre>if (u4t_x == 10) { } while (s2t_y != 0){ }</pre>
18	Control flow	Rule 18 The body of an iteration-statement or a selection-statement shall be a compound-statement	The body of an iteration-statement (while, do ... while or for) or a selection-statement (if, else, switch) shall be a compound-statement. Non-compliant example <pre>if (u4t_x < 30) action1(); else action2(); while (u4t_y != 20) action3();</pre> Compliant example <pre>if (u4t_x < 30) { action1(); } else { action2(); } while (u4t_y != 20) { action3(); }</pre>

Check Items			Example / Amplification
No.	Class	Description	
19	Control flow	Rule 19 All if ... else if constructs shall be terminated with an else statement	Non-compliant example <pre>if (u4t_x < 10) { action1(); }</pre> <- Don't have else statement <pre>if(u4t_y > 0) { action2(); } else if (u4t_y <0) { action3(); }</pre> <- Don't have else statement Compliant example <pre>if (u4t_x < 10) { action1(); } else { /* No action */ }</pre> <pre>if(u4t_y > 0) { action1(); } else if (u4t_y <0) { action2(); } else { /* Do nothing */ }</pre>
20	Switch Statements	Rule 20 An unconditional break statement shall terminate every switch-clause	Non-compliant example <pre>switch (u1t_x) { case 0: <- Compliant - unconditional break action1(); break; case 1: <- Compliant - empty fall through allows a group case 2: action2(); break; case 3: <- Non-compliant - break omitted action3(); case 4: <- Non-compliant - conditional break if(action4() > 20) { action4(); break; } else { /* No action */ } default: <- Non-compliant - Default must also have a break }</pre>

Check Items			Example / Amplification
No.	Class	Description	
21	Switch Statements	Rule 21 Every switch statement shall have a default label	Non-compliant example <pre>switch (u1t_x) { case 0: action1(); break; case 1: action2(); break; }</pre> <- Non-compliant - No default case Compliant example <pre>switch (u1t_x) { case 0: action1(); break; case 1: action2(); break; default: action3(); break; }</pre> <- Compliant
22		Rule 22 A default label shall appear as either the first or the last switch label of a switch statement	Non-compliant example <pre>switch (u1t_x) { case 0: action1(); break; default: action3(); break; case 1: action2(); break; }</pre> <- Non-compliant Compliant example <pre>switch (u1t_x) { case 0: action1(); break; case 1: action2(); break; default: action3(); break; }</pre> <- Compliant - default is the last label